

Case Swarm Chat Bridge

Options, Limits and Cost Reference

Generated 2026-07-23

Current path: Azure live chat bridge (LWC polls Azure ~every 2s)

Revert: Teams_Bot_Config.Use_Azure_Chat_Bridge__c = false -> Apex Graph polling

Reference for choosing and operating the Salesforce <-> Microsoft Teams chat bridge. Source markdown: docs/Case-Swarm-Chat-Bridge-Options-and-Limits.md

1. Requirement (what we optimized for)

Priority	Requirement
Must	Near-real-time chat: agent (Salesforce LWC) <-> SME (Teams)
Must	Chat swarms only (not Team/channel)
Nice	No Salesforce record required per message (Swarm_Message__c deferred)
Constraint	Protect Salesforce Daily REST API and Apex HTTP callouts
Constraint	Also care about Azure Function executions / cost
Constraint	Outbound posts as the bot (Graph app-only cannot send live chat)

Hard truth: every design spends somewhere (Salesforce, Azure, Graph, or UX delay). There is no free real-time path.

2. Architecture map (current)

```
Teams chat
|- install / Adaptive Card invoke
|   -> teamsBotRelay -> Salesforce TeamsBotMessagingResource
|- plain type=message
|   -> teamsBotRelay returns 200 (no Salesforce call)

Salesforce LWC (caseSwarmChatCore)
|- once: getChatBridgeSession (Apex HMAC token)
|- ~2s: GET Azure /api/chatHistory -> Microsoft Graph list messages
|- send: POST Azure /api/chatSend -> Bot Framework Connector
```

Teams is the source of truth for message history in the current path (B). No Azure Table/Cosmos is used today. Option G (Azure Table Storage) is documented as an alternative buffer.

Feature flag

CMDT field	Effect
Use_Azure_Chat_Bridge__c = true	LWC -> Azure history/send
Use_Azure_Chat_Bridge__c = false	LWC -> Apex syncMessages / sendMessage (original Graph poll)

Also required when Azure is on: Chat_Bridge_Azure_Base_Url__c, Chat_Bridge_HMAC_Secret__c (match Azure CHAT_BRIDGE_HMAC_SECRET), CSP Trusted Site for the Function App URL. See README section 6h.

3. Options catalogue

A. Apex Graph poll (original)

LWC every ~5s -> Apex syncMessages -> Graph; send via Apex Bot callout; optional Swarm_Message__c.

SF	Azure	Live	Complexity
High callouts while open	Low	~5s	Low

B. Azure live poll (stateless) - CURRENT

LWC every ~2s -> Azure chatHistory -> Microsoft Graph list; send via Azure chatSend -> Bot; plain messages not forwarded to SF. No Azure Table - Teams/Graph is the only message store.

SF	Azure	Live	Complexity
Very low for chat	High if panels stay open (Graph every poll)	~0-2s	Low-medium

C. Azure poll tuned (5-10s / focus only)

Same as B with slower or smarter polling.

SF	Azure	Live	Complexity
Very low	Medium	OK	Low

D. SignalR / push (recommended next at scale)

Teams message -> relay -> Azure SignalR -> LWC; send via Azure Bot; optional one-time Graph history on open. Prefer SignalR over raw SSE on Functions (HTTP ~230s limit).

SF	Azure	Live	Complexity
Very low	Low (per message + connections)	Best	Medium

E. Per-message into Salesforce

Every Teams message -> Apex REST; every SF send -> Apex callout; optional Platform Events.

SF	Azure	Live	Complexity
High (linear with messages)	Low	Best	Medium

F. Graph change-notification webhooks

Graph POSTs to Azure on new messages; then push (D), Table (G), or SF write (E).

SF	Azure	Live	Complexity
Depends	Medium (subscriptions)	Best	High

G. Azure Table Storage (message buffer)

Bot/webhook upserts each message into Azure Table (PartitionKey=chatId). LWC reads Table (cheap) instead of Graph every 2s. Optional batch to Swarm_Message__c on close / timer. Variants: G1 live Table only; G2 Table+batch to SF; G3 Table+SignalR. Not built in current repo (current=B).

SF	Azure	Live	Complexity
Very low (unless batch every msg)	Medium (Table writes + cheap reads)	Near RT / instant w/ SignalR	Medium-higher

Option G flow

Teams message -> bot relay (or Graph webhook)
 -> upsert Azure Table (PartitionKey = chatId)
 SF send -> chatSend -> Bot Framework
 -> upsert Outbound in same table
 LWC -> GET history from Table (not Graph every 2s)
 Optional batch -> timer / on close -> Swarm_Message__c

B Stateless Graph poll	G Azure Table
History each poll: Graph list	History each poll: Table query
Inbound write: none	1 Table upsert per Teams message
Graph while open: every ~2s	Graph rare / never
Graph blip: history fails	LWC can read last stored rows

H. Sync on open / close / schedule only

No live poll; one Graph sync when needed.

SF	Azure	Live	Complexity
Very low	Very low	No	Low

I. SOQL-only UI poll

Poll listStoredMessages only; another writer must create rows.

SF	Azure	Live	Complexity
SOQL load	Depends on writer	Only if rows appear	Low

J. Teams-only (no SF chat UI)

Case has Open in Teams + swarm metadata only.

SF	Azure	Live	Complexity
Minimal	Minimal	In Teams only	Lowest

K. Salesforce Messaging / BYOC

Full Service Cloud messaging channel.

SF	Azure	Live	Complexity
Platform	Adapter	Native Omni	Very high

4. Ranking for this requirement

Optimized for: live chat + low SF + low Azure + reasonable complexity.

Rank	Option	Verdict
1	D - SignalR push	Most efficient / cost-effective at scale
2	B - Azure poll ~2s (current)	Best simplicity / value now; burns Azure/Graph while idle
3	C - Slower / focus-only Azure poll	Cheap interim if Azure \$ grows
4	G - Azure Table (+ optional SignalR/batch)	Cuts Graph poll cost; adds storage; good for history buffer / SF transcript
5	E - Per-message Salesforce API	Good for Azure; bad for SF
6	A - Apex Graph poll	Worst for SF callouts
7	H - Sync on open/close	Cheapest; fails live requirement
8	K - Messaging BYOC	Overkill for Case swarm widget

One-line recommendation

- Today: keep Azure poll (B).
- If Azure/Graph poll volume hurts: prefer SignalR (D) first; use Table (G) if you also need Azure history or batch transcript to SF.
- Do not go to per-message Salesforce (E) unless SF limits no longer matter.
- Do not return to Apex polling (A) for live chat.

5. Worked example - limits comparison

Assumptions: Active Chat swarm, panel open 45 minutes, 25 messages (15 from Teams, 10 from Salesforce), bot already installed.

Meter	A Apex 5s	B Azure 2s	D SignalR	G Table poll	E SF/msg
SF Daily REST API	~1-2	~1-2	~1-2	~1-2 (+batch)	~15-17
Apex callouts (chat)	~540+	~0	~0	~0	~10-13
Azure Function ops	Low	~1,300+ hist	~25-40	~1,300 reads+25 writes	~25
Graph list open	~540 SF	~1,300 Azure	~0-1	~0	0
Table transactions	0	0	0	~1,300+25	0
Live UX	~5s	~0-2s	Instant	~0-2s / instant	Instant

Note: G still has many Function executions if LWC polls Table every 2s; combine with SignalR (G3) to drop reads to about per message.

6. What is / isn't a Salesforce callout or API limit

Hop	SF Daily REST API?	Apex HTTP callout?
LWC -> Apex syncMessages	No	Yes (to Graph)
LWC -> Azure chatHistory (browser fetch)	No	No
Azure -> Graph	No	No (Microsoft Graph)
Azure -> Bot Framework	No	No
Relay -> Apex REST (bot install / card)	Yes	No (inbound)
LWC -> Apex sendMessage -> Bot	No	Yes
LWC -> Azure chatSend -> Bot	No	No

7. Azure Functions limits (summary)

See Microsoft Learn: Azure Functions scale and hosting.

Resource	Consumption (legacy)	Flex Consumption
Default timeout	5 min	30 min
Max timeout	10 min	Unbounded*
HTTP response hard cap	~230 seconds (all plans)	Same
Memory / instance	1.5 GB	512 MB / 2 GB / 4 GB
Max instances	Win 200 / Linux 100	Up to ~1,000 (regional quota)
Outbound connections / instance	600 active	High / unbounded

*HTTP triggers still bound by the ~230s load-balancer limit. Classic Consumption free grant (verify current pricing): ~1M executions/month and ~400k GB-seconds, then pay-per-use.

Poll math: 1 open panel @ 2s ~ 30 executions/min ~ 43k/day. Ten panels x 8 hours ~ 144k/day. What fails first at growth: often Microsoft Graph throttling (429) and Function execution cost.

8. SignalR path - what's needed (upgrade)

Need	Purpose
Azure SignalR Service (Serverless)	Hold live connections
negotiate Function	LWC gets SignalR URL + token
Update teamsBotRelay	On type=message, push to SignalR group = chatId
Keep chatSend	SF -> Bot; optional SignalR echo
Optional one-shot chatHistory	Load last N on open only
LWC SignalR client	Subscribe; remove/slow 2s poll
CSP for SignalR + Function hosts	Browser connect-src
CMDT mode flag	e.g. Poll vs SignalR; keep poll fallback

Not required for live MVP: message Table storage, Graph webhooks, SF Platform Events, per-message Apex. Do not rely on raw long-lived SSE from a Consumption/Flex HTTP Function alone (230s cut-off).

9. Enable / revert (operations)

Enable Azure bridge (current)

1. Deploy Salesforce metadata + relay/ Function App.
2. Set Azure app settings: Bot + Graph + CHAT_BRIDGE_HMAC_SECRET (+ CORS).
3. Set CMDT: Azure base URL + HMAC secret.
4. Update CSP Trusted Site Chat_Bridge_Azure_Connect to the real Function URL.
5. Set Use_Azure_Chat_Bridge__c = true.
6. Smoke-test SF <-> Teams.

Revert to Apex Graph polling

Set Use_Azure_Chat_Bridge__c = false (or clear Azure URL/secret). No code rollback required. LWC uses syncMessages / sendMessage again.

10. Decision cheat sheet

If you care most about...	Choose
---------------------------	--------

If you care most about...	Choose
Shipping / ops simplicity	B - Azure poll (current)
Azure \$ with many open panels	D - SignalR (or C as quick win)
Cut Graph list cost but keep poll UI	G - Azure Table (history from Table)
Live chat + later Case transcript	G2 - Table + batch to Swarm_Message__c
Best Azure efficiency + history buffer	G3 - Table + SignalR
Salesforce API / callouts	B, D, or G (avoid A and E)
Live not required	H
Full Service Console messaging	K

11. Related code & docs

Path	Role
relay/.../teamsBotRelay.js	Bot -> SF for cards; ack-only for plain messages
relay/.../chatHistory.js	Azure -> Graph history
relay/.../chatSend.js	Azure -> Bot send
lwc/caseSwarmChatCore/	UI; Azure vs Apex mode
TeamsSwarmChatController.cls	getChatBridgeSession, legacy sync/send
Teams_Bot_Config__mdt	Bridge flag, URL, HMAC
README.md 6f / 6h	Relay + Azure chat bridge setup

12. Glossary

Term	Meaning
Callout	Apex HTTP request from Salesforce to an external system
Daily REST API	Inbound REST/SOAP to Salesforce (e.g. relay -> Apex REST)
Stateless proxy	Azure does not store chat; Teams/Graph holds history (current B)
Azure Table Storage	Cheap key-value chat buffer (G); PartitionKey = chatId
SignalR	Azure push service for browsers; preferred over raw SSE on Functions